

Kafka: Paper Review

Rohan Gore - rmg9725

Collaborators: ChatGPT 5

References

1. “Kafka: a Distributed Messaging System for Log Processing” (original paper)

1 Problem Statements Tackled

1. Large-scale web platforms generate massive streams of user activity and operational metrics that must support both real-time production use cases (search relevance, recommendations, ad targeting, security) and offline analytics simultaneously.
2. Existing solutions like enterprise messaging systems and log aggregators, either lacked the throughput, scalability, and distributed design required, or provided only bulk offline ingestion without real-time, multi-subscriber consumption semantics.
3. LinkedIn required a unified system capable of durable log storage, horizontal scalability, low-latency streaming consumption, and the ability for independent consumers to replay, rewind, and process data at arbitrary speeds.

2 Key Design Principles

1. **Log-Centric, Append-Only Storage Model**
Kafka models each topic as a distributed commit log split into partitions. Messages are appended sequentially to log segment files, enabling high-throughput writes, efficient disk access patterns, and OS-optimized caching behavior. Consumers track their own read offsets, eliminating the need for broker-side delivery state.
2. **Simple, Minimal Messaging API**
Kafka exposes a lightweight producer/consumer API. Producers batch messages and publish them as raw byte payloads. Consumers iterate over infinite message streams that block when no new data is available, minimizing API complexity and reflecting the continuous nature of log data.
3. **Pull-Based Consumption with Consumer-Managed Offsets**
Consumers fetch data via explicit pull requests and maintain their own offsets, enabling natural backpressure control, rewind/replay functionality, and independence between consumer groups. This avoids queue buildup problems typical of push-based systems.
4. **OS Page Cache and Zero-Copy I/O for Throughput**
Kafka avoids application-level caching and relies on the file system page cache for buffering. Combined with Linux’s `sendfile()` zero-copy transfer, Kafka minimizes redundant copying, reduces GC pressure, and achieves near-optimal sequential disk and network throughput.

3 Architecture and Components

3.1 Partitioned Log Storage and Single-Partition Efficiency

A partition is implemented as a logical log composed of fixed-size segment files. Producers append new messages to the active segment, which is flushed to disk after a configurable batch size or timeout. Each message is addressed by a logical offset. Brokers store an in-memory index mapping segment boundaries to offsets, enabling efficient lookup of the segment containing a requested offset. Consumers issue fetch requests specifying a starting offset and byte limit; they compute the next offset by adding the size of the consumed message, ensuring strictly sequential consumption. Kafka’s use of sequential disk access, page-cache readahead, and write-back policies provides performance that scales linearly with data size.

3.2 Producer and Consumer Interaction Across Brokers

Producers publish messages to topic partitions using deterministic (key-based) or round-robin routing. Consumers create message streams for subscribed topics, with messages evenly distributed into sub-streams. Kafka supports both point-to-point delivery via consumer groups—where each partition is consumed by only one consumer—and classical publish/subscribe, where each group independently reads the full log. Message stream iterators block when no data is available, reflecting the continuous arrival of events.

3.3 Zero-Copy Reads and Efficient Network Transfer

To support multi-subscriber fan-out, Kafka uses Linux’s `sendfile()` API, allowing the kernel to transfer bytes from a file descriptor directly to a socket without routing data through user-space buffers. This eliminates several memory copies and system calls, significantly increasing throughput for consumers reading large, contiguous segments of the log.

3.4 Stateless Broker and Retention Semantics

Kafka brokers do not track consumer positions, reducing system complexity and eliminating per-consumer metadata overhead. Because consumers may progress at different rates, Kafka relies on time-based retention to delete old messages. Logs are typically retained for days, enabling offline consumers (e.g., Hadoop ETL jobs) to process data in periodic batches. The stateless model also enables consumer rewind: a consumer can process data from an earlier offset when recovering from failures or reloading historical state.

3.5 Distributed Coordination via ZooKeeper

Kafka uses ZooKeeper to manage cluster metadata: broker registration, consumer membership, ownership of partitions, and committed offsets. ZooKeeper ephemeral nodes provide automatic cleanup on failure, and watchers notify consumers of broker or membership changes. During a rebalance, consumers read broker and consumer registries, compute the global assignment of partitions, and create ephemeral ownership nodes. Conflicts are resolved through retries, and consumers start fetching from the last committed offsets.

4 Related Work

1. **Enterprise Messaging Systems:** Traditional platforms such as JMS and WebSphere MQ emphasize strong delivery guarantees (transactions, per-message acknowledgments), creating overhead unsuited for high-throughput log ingestion.
2. **Log Aggregators (Scribe, Data Highway, Flume):** Designed for bulk offline ingestion into HDFS/NFS, these systems expose internal batching artifacts, provide limited real-time consumption semantics, and rely on push-based delivery. Flume still remains optimized for offline usage.
3. **Distributed Pub/Sub Research Systems:** Systems like HedWig offer scalable, durable pub/sub messaging but primarily target commit-log storage for distributed databases. Kafka, by contrast, unifies durable log storage, horizontal partitioning, multi-subscriber consumption, and real-time streaming in a single architecture supporting both online and offline processing.

5 Conclusion

Kafka introduces a unified messaging and log-processing architecture optimized for high-throughput, horizontally scalable, multi-subscriber data pipelines. By adopting an append-only partitioned log abstraction, Kafka delivers strong sequential ordering within partitions, efficient persistent storage, and low-latency streaming access. Its reliance on the OS page cache, zero-copy network transfer, and consumer-managed offsets enables consistent performance at terabyte scale. ZooKeeper-based coordination provides decentralized membership and ownership management, supporting elastic scaling and fast failover. Kafka’s design has significantly simplified LinkedIn’s infrastructure by serving as a single substrate for both real-time stream processing and batch data ingestion, defining a model that influenced the next generation of distributed streaming systems.